We iterate over the returned node list, and for all matching nodes with a type matching `XML_ELEMENT_NODE`, we then iterate over those element's `attributes` property (which is itself another node list). Hopefully by now you're starting to get a feel for this: nodes represent nearly *everything* in the DOM (from the entire document down to a single element's attributes), and consequently `Dom\Node` and `Dom\NodeList` are the basic unit of currency.

`class` and `id`

As mentioned above, `class` and `id` attributes are special. They provide mechanisms for identifying elements individually and for grouping elements together that might otherwise be in very different places in the DOM. Both JavaScript and CSS are heavily dependent on these attributes: the former for hooking behavior to the DOM, the latter for applying rendering rules. When classes and ids are applied to elements, they become much easier to find, and the `dom` extension offers a powerful tool for doing just that. Let's tweak our sample HTML a little to demonstrate:

```
$validHtml = <<<EOS
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>Block level element demo</title>
  </head>
  <body>
    <main>
      <h1>Top-level heading</h1>
      <p id="first_para">A simple demo of block level elements</p>
      <div>
        &#60;hgroup&#62; elements were deprecated in the HTML5 spec
      </div>
      <p class="another_para">...continuing demo...</p>
      <p class="another_para">End of the demo</p>
    </main>
  </body>
</html>
EOS;
```